

Trilha Segurança Full Stack — Plano técnico para nível avançado/sênior

Plano técnico estruturado para evolução de fundamentos até domínio sênior, com foco em AppSec, DevSecOps, cloud, Kubernetes, detecção, resposta e supply chain security.

Formato	Documento técnico em PDF
Objetivo	Formar visão de Security Champion aplicada a projetos full stack
Escopo	Frontend, backend, APIs, DevSecOps, cloud, containers, SIEM e supply chain
Fonte	Texto colado(68).txt

Como usar este material

Leia primeiro a análise inicial, avance pelos módulos em ordem e use os critérios de domínio como checklist de progressão. O foco é aplicar segurança em projeto real, com decisões justificáveis em ambiente de produção.

Sumário executivo

Principais blocos do conteúdo anexado, organizados para navegação rápida.

1. Análise inicial da trilha	3
2. Classificação por nível	3
3. Explicação técnica dos tópicos	4
4. Plano de estudos por módulos	14
5. Critérios de domínio técnico por grande área	22
6. Aplicação prática em projetos reais	23
7. Cronograma sugerido — 20 semanas	26
8. Projetos recomendados para portfólio	30
9. Checklist de domínio técnico	31
10. Indicadores de prontidão para nível avançado/sênior	33

1. Análise inicial da trilha

Tema central

A trilha trata de segurança aplicada ao desenvolvimento full stack, cobrindo desde fundamentos de AppSec até autenticação, autorização, segurança frontend/backend, DevSecOps, cloud, Kubernetes, pentest controlado, detecção, resposta a incidentes e supply chain security. O foco não é formar apenas alguém que “conhece vulnerabilidades”, mas sim um Security Champion: um desenvolvedor capaz de antecipar riscos, revisar arquitetura, implementar controles, automatizar verificações e defender aplicações em produção. OWASP Top 10 continua sendo uma referência central para riscos críticos em aplicações web; OWASP API Security Top 10 cobre riscos específicos de APIs, como BOLA e BFLA; ASVS fornece requisitos verificáveis para segurança de aplicações; NIST SSDF organiza práticas de desenvolvimento seguro no ciclo de vida de software.

Objetivo principal

Formar um profissional capaz de:

- Projetar aplicações seguras por padrão.
- Reduzir superfície de ataque.
- Implementar autenticação e autorização robustas.
- Evitar falhas comuns como XSS, CSRF, SQL Injection, SSRF, IDOR, BOLA e mass assignment.
- Integrar segurança em pipelines CI/CD.
- Proteger containers, imagens, dependências e ambientes cloud.
- Criar relatórios técnicos de vulnerabilidade.
- Interpretar logs, eventos e incidentes.
- Tomar decisões arquiteturais baseadas em risco, impacto, custo e contexto.

2. Classificação por nível

Básico

O estudante deve entender:

- CIA Triad: confidencialidade, integridade e disponibilidade.
- Diferença entre risco, vulnerabilidade, ameaça e exploit.
- OWASP Top 10.
- Conceitos de autenticação, autorização, sessão, token e senha.
- Noções de XSS, CSRF, CORS, SQL Injection e exposição de dados.

Intermediário

O estudante deve aplicar:

- Validação de entrada.
- Sanitização e encoding de saída.
- JWT com boas práticas.
- Cookies seguros.
- Rate limiting.
- RBAC.
- Hash de senha com bcrypt ou Argon2.

- Segurança em APIs REST.
- SAST, DAST e SCA em pipeline.
- Hardening básico de containers.

Avançado

O estudante deve dominar:

- Threat modeling.
- OAuth2, OIDC, PKCE, claims, scopes e refresh token rotation.
- ABAC e autorização contextual.
- Segurança de GraphQL.
- DevSecOps.
- Kubernetes RBAC, NetworkPolicy e Pod Security.
- WAF, IAM, KMS, Secrets Manager e segmentação de rede.
- SBOM, assinatura de imagem e política de deploy.

Sênior

O profissional sênior deve decidir e justificar:

- Qual modelo de autenticação usar.
- Quando usar session, JWT, opaque token ou token introspection.
- Como separar responsabilidades entre frontend, backend, gateway e IdP.
- Como proteger o ciclo código → build → imagem → deploy → runtime.
- Como priorizar vulnerabilidades por impacto real.
- Como balancear segurança, custo, performance, DX e time-to-market.
- Como transformar segurança em processo contínuo, não em etapa final.

3. Explicação técnica dos tópicos

3.1 Fundamentos de segurança

O que estudar

CIA Triad, threat modeling, attack surface, risk, vulnerability, exploit, defense in depth, least privilege, secure by default, secure by design, data classification, LGPD e privacy by design.

O que é

Fundamentos são a base conceitual para entender por que uma aplicação precisa de controles de segurança. Sem isso, o desenvolvedor apenas copia práticas sem entender contexto.

Para que serve

Serve para avaliar riscos antes da implementação. Por exemplo: um endpoint de consulta de pedidos pode parecer simples, mas pode expor dados pessoais, permitir IDOR, vaziar informações em logs ou permitir abuso por automação.

Onde é usado em projetos reais

- Desenho de APIs.
- Definição de permissões.
- Modelagem de banco.

- Classificação de dados sensíveis.
- Escolha de logs.
- Definição de controles em cloud.
- Revisão de arquitetura.
- Priorização de correções.

Problemas que resolve

- Exposição indevida de dados.
- Permissões excessivas.
- Falhas de desenho.
- Vazamento de credenciais.
- Falta de rastreabilidade.
- Soluções inseguras por padrão.

Erros comuns

- Tratar segurança como checklist final.
- Não classificar dados.
- Dar permissão ampla “para funcionar”.
- Logar dados sensíveis.
- Não revisar fluxos de abuso.
- Confundir autenticação com autorização.

Profundidade sênior

Um sênior deve saber transformar requisitos técnicos em controles defensivos proporcionais ao risco. Nem todo sistema exige o mesmo nível de proteção, mas todo sistema precisa de uma justificativa clara para suas decisões.

3.2 OWASP Top 10

O que estudar

Broken Access Control, Cryptographic Failures, Injection, Insecure Design, Security Misconfiguration, Vulnerable Components, Authentication Failures, Software and Data Integrity Failures, Logging and Monitoring Failures e SSRF.

O que é

OWASP Top 10 é uma referência de conscientização sobre os principais riscos de segurança em aplicações web. A versão atual de referência para web apps é a OWASP Top 10 2021, enquanto APIs possuem uma lista própria no OWASP API Security Top 10 2023.

Para que serve

Serve como mapa inicial para revisar código, arquitetura e configuração. Não substitui ASVS, threat modeling ou pentest, mas ajuda a identificar riscos recorrentes.

Onde é usado

- Code review.
- Security review.
- Testes automatizados.

- Políticas de pipeline.
- Checklists de PR.
- Treinamento de times.
- Critérios mínimos para produção.

Problemas que resolve

- Falhas previsíveis de autorização.
- Injeções.
- Criptografia mal aplicada.
- Componentes vulneráveis.
- Configuração insegura.
- Falta de logs úteis.
- Falhas de design.

Erros comuns

- Decorar o Top 10 sem saber corrigir.
- Focar só em SQL Injection e ignorar autorização.
- Achar que WAF corrige falha de código.
- Ignorar dependências transitivas.
- Não testar autorização por objeto.

Profundidade sênior

O sênior precisa conseguir olhar um endpoint e perguntar:

- Quem pode acessar?
- A permissão é validada no backend?
- O usuário pode alterar o ID?
- O dado retornado é necessário?
- Há logs de tentativa negada?
- O erro expõe detalhe interno?
- O endpoint pode ser abusado em escala?

3.3 Autenticação e autorização

O que estudar

Sessions, JWT, refresh token rotation, OAuth2, OIDC, PKCE, scopes, claims, RBAC, ABAC, MFA, hashing de senha, Argon2, bcrypt, rate limiting, account lockout e token revocation. OAuth2 é um protocolo de autorização amplamente usado, enquanto OIDC adiciona camada de identidade sobre OAuth2. PKCE é especialmente relevante para fluxos modernos com clientes públicos e aplicações frontend.

O que é

- Autenticação responde: quem é o usuário?
- Autorização responde: o que esse usuário pode fazer?

Aplicação real

Em um sistema como a DevCore:

- Login com usuário e senha.
- Senha armazenada com Argon2id ou bcrypt.
- Access token curto.
- Refresh token rotativo.
- Cookie HttpOnly/Secure/SameSite.
- CSRF token quando usar cookie autenticado.
- MFA para operações sensíveis.
- Revogação de sessão.
- Auditoria de login e refresh. OWASP recomenda algoritmos modernos e apropriados para armazenamento de senha, com parâmetros de custo adequados ao ambiente.

Erros comuns

- Guardar JWT em LocalStorage sem avaliar risco de XSS.
- JWT sem expiração curta.
- Refresh token sem rotação.
- Senha com hash fraco.
- Usar MD5, SHA-1 ou SHA-256 puro para senha.
- Permissão validada apenas no frontend.
- Confiar cegamente em claims sem validação.
- Não revogar tokens em logout, troca de senha ou suspeita de vazamento.

Profundidade sênior

Um sênior deve saber escolher entre:

- Session tradicional.
- JWT stateless.
- JWT com denylist.
- Opaque token com introspection.
- Autorização centralizada via gateway.
- Autorização distribuída por serviço.
- RBAC simples.
- ABAC contextual. A decisão depende de escala, criticidade, necessidade de revogação, arquitetura, UX, custo operacional e risco.

3.4 Segurança frontend

O que estudar

XSS, CSP, CORS, CSRF, clickjacking, secure cookies, riscos de LocalStorage, dependências, validação de entrada, output encoding, upload seguro e security headers no Next.js.

O que é

Segurança frontend reduz riscos no navegador, especialmente quando o usuário interage com conteúdo dinâmico, cookies, tokens, formulários, uploads e scripts de terceiros. XSS pode permitir execução de conteúdo malicioso no contexto da aplicação; CSRF força o navegador autenticado a executar uma ação indesejada em uma aplicação vulnerável.

Aplicação real

Em Next.js ou React:

- Evitar renderização de HTML não confiável.
- Usar encoding correto.
- Configurar CSP.
- Configurar X-Frame-Options ou frame-ancestors.
- Usar cookies HttpOnly, Secure e SameSite.
- Validar arquivos no frontend e backend.
- Evitar expor secrets em NEXT_PUBLIC_*.
- Revisar bibliotecas de terceiros.

Erros comuns

- Achar que React elimina todo XSS automaticamente.
- Usar dangerouslySetInnerHTML sem sanitização.
- Colocar access token em LocalStorage sem mitigação.
- Configurar CORS com * em APIs autenticadas.
- Aceitar upload apenas validando extensão.
- Expor chaves privadas no bundle frontend.

Profundidade sênior

O sênior deve avaliar:

- Modelo de armazenamento de token.
- Política de CSP viável sem quebrar produto.
- Fronteira entre validação frontend e backend.
- Segurança de bibliotecas.
- Impacto de scripts externos.
- Proteção contra abuso de formulários e uploads.

3.5 Segurança backend e API

O que estudar

Input validation, output encoding, SQL Injection, NoSQL Injection, SSRF, IDOR, BOLA, BFLA, mass assignment, rate limiting, request size limit, error handling, logging seguro, secrets, API Gateway e GraphQL security.

O que é

Segurança backend protege regras de negócio, dados, integrações e autorização real. O backend é o ponto onde as decisões de confiança devem ser efetivadas. No OWASP API Security Top 10 2023, BOLA é destacado porque APIs frequentemente expõem identificadores de objetos e exigem validação de autorização em cada acesso por objeto.

Aplicação real

Em uma API Java/Spring:

- Validar DTOs com Bean Validation.
- Usar queries parametrizadas.
- Validar autorização por recurso.
- Bloquear mass assignment com DTOs específicos.
- Limitar tamanho de payload.
- Usar rate limit por IP, usuário e rota.
- Tratar erros sem expor stack trace.
- Não logar token, senha, CPF, cartão ou secrets.
- Isolar chamadas externas para reduzir SSRF.
- Usar allowlist para URLs internas quando necessário.
- Proteger GraphQL com limite de profundidade e complexidade.

Erros comuns

- Endpoint `/users/{id}` sem validar dono do recurso.
- Retornar entidade JPA diretamente.
- Permitir update genérico de qualquer campo.
- Usar `Map<String, Object>` sem validação.
- Expor detalhes técnicos em erro.
- Logar payload completo em produção.
- Não limitar upload e request body.
- Não proteger endpoints internos.

Profundidade sênior

O sênior deve revisar contrato de API, modelo de autorização, fluxo de dados, logs, limites, tolerância a abuso e impacto em produção. Segurança de API não é apenas “validar token”; é garantir que cada operação respeita identidade, permissão, contexto e integridade do domínio.

3.6 DevSecOps

O que estudar

SAST, DAST, SCA, IAST conceitual, SBOM, secret scanning, dependency scanning, container scanning, IaC scanning, SonarQube, Trivy, Semgrep, OWASP ZAP, Dependency-Check e GitHub Advanced Security conceitual.

O que é

DevSecOps integra segurança no ciclo de desenvolvimento, reduzindo o custo de correção e aumentando a visibilidade de risco. NIST SSDF recomenda práticas de desenvolvimento seguro que podem ser incorporadas ao SDLC, enquanto OWASP ASVS oferece requisitos verificáveis para controles técnicos de aplicação.

Aplicação real

Pipeline mínimo:

- Secret scanning no commit.
- SAST no pull request.
- SCA em dependências.
- Testes unitários e integração.
- Build de imagem.
- Container scanning.
- SBOM gerado.
- Assinatura de imagem.
- Policy gate antes do deploy.
- DAST em ambiente controlado.

Erros comuns

- Rodar ferramenta sem política de bloqueio.
- Ignorar findings críticos.
- Tratar falso positivo como desculpa para não corrigir.
- Não versionar exceções.
- Não definir SLA de vulnerabilidade.
- Não escanear imagem final.
- Não validar IaC.

Profundidade sênior

O sênior deve definir:

- O que bloqueia build.
- O que vira backlog.
- SLA por severidade.
- Política de exceção.
- Evidência para auditoria.
- Métricas de segurança.
- Como reduzir ruído sem perder risco real.

3.7 Segurança cloud e Kubernetes

O que estudar

IAM, least privilege, SCPs, permission boundaries, Secrets Manager, KMS, segmentação de rede, security groups, WAF, GuardDuty, Security Hub, Kubernetes RBAC, Pod Security Admission, NetworkPolicy, OPA/Gatekeeper, Falco, audit logs e image signing. Kubernetes RBAC regula acesso a recursos por papéis; Pod Security Standards definem níveis de política para segurança de Pods; NetworkPolicy e políticas de admissão ajudam a reduzir superfície de ataque no cluster.

Aplicação real

Em produção:

- Cada serviço com service account própria.
- IAM mínimo por workload.
- Secrets fora do repositório.
- Chaves gerenciadas por KMS.
- Rede segmentada por ambiente.
- Pods sem privilégio.
- Imagens assinadas.
- Deploy bloqueado se imagem não for confiável.
- Logs de auditoria habilitados.
- WAF protegendo borda.
- Alertas para comportamento suspeito.

Erros comuns

- Usuário cloud com permissão admin.
- Secret em variável exposta ou repositório.
- Security group aberto para internet.
- Container rodando como root.
- Pod com privilégios excessivos.
- Cluster sem NetworkPolicy.
- Imagem sem pinning.
- Falta de auditoria.

Profundidade sênior

O sênior precisa projetar segurança em camadas:

- Identidade.
- Rede.
- Workload.
- Imagem.
- Runtime.
- Logs.
- Políticas.

- Resposta a incidente. A decisão correta depende do risco do sistema, criticidade dos dados, maturidade da equipe, custo cloud e capacidade operacional.

3.8 Pentest básico controlado

O que estudar

Recon básico, OWASP ZAP, Burp Suite CE, teste de autenticação, teste de autorização, IDOR, SSRF, XSS em laboratório, SQL Injection em laboratório, misconfiguration e report de vulnerabilidade.

Escopo seguro

Esse módulo deve ser feito somente em aplicação própria, ambiente local, laboratório autorizado ou ambiente de teste formalmente permitido. O objetivo é aprender a validar defesa, não atacar terceiros.

Entregável

Relatório de pentest da DevCore contendo:

- Vulnerabilidade.
- Impacto.
- Evidência.
- Severidade.
- Risco de negócio.
- Correção recomendada.
- Responsável.
- Prazo.
- Reteste.
- Status final.

Erros comuns

- Testar ambiente sem autorização.
- Focar em “achar falha” e não em corrigir.
- Não reproduzir evidência.
- Não classificar severidade.
- Não fazer reteste.
- Não transformar achados em melhoria de arquitetura.

Profundidade sênior

O sênior deve saber interpretar vulnerabilidade como risco de negócio. Uma falha técnica só é bem tratada quando existe contexto: impacto, explorabilidade, exposição, criticidade do ativo e custo de correção.

3.9 SIEM, detecção e resposta

O que estudar

Logs de segurança, correlation rules, detection engineering, MITRE ATT&CK, false positives, alert tuning, Splunk conceitual, ELK/OpenSearch, incident response, playbooks e forensics básico. MITRE ATT&CK é uma base de conhecimento de táticas e técnicas adversárias baseada em observações reais, usada para modelagem de ameaças, detecção e metodologias defensivas. NIST SP 800-61 Rev. 3 orienta resposta a incidentes integrada ao NIST CSF 2.0.

Aplicação real

Eventos úteis:

- Login falho repetido.
- Refresh token reutilizado.
- Tentativa de acesso negado.
- Alteração de papel/permissão.
- Upload bloqueado.
- Mudança de senha.
- Criação de chave de API.
- Erro de validação incomum.
- Pico de 403/429.
- Acesso fora de padrão.

Erros comuns

- Logar demais sem utilidade.
- Logar dados sensíveis.
- Não ter correlation ID.
- Não ter playbook.
- Alertas sem prioridade.
- Ignorar falso positivo até o time parar de olhar.
- Não fazer pós-incidente.

Profundidade sênior

O sênior deve conectar aplicação, infraestrutura e negócio:

- Quais eventos indicam abuso?
- Qual alerta exige ação imediata?
- Qual alerta pode virar métrica?
- Qual log ajuda investigação?
- Qual dado não deve ser logado?
- Como reduzir ruído?
- Como melhorar o playbook após incidente?

3.10 Supply chain security

O que estudar

SBOM, SLSA, Sigstore, Cosign, in-toto, provenance, image signing, dependency pinning, reproducible builds, artifact registry e policy gates. SLSA é um framework para melhorar segurança da cadeia de suprimentos de software, com controles contra adulteração e para integridade de pacotes e infraestrutura. Sigstore facilita assinatura e verificação de componentes de software.

Aplicação real

Fluxo seguro:

- Código versionado.

- Dependências fixadas.
- Build reproduzível.
- SBOM gerado.
- Artefato publicado em registry.
- Imagem assinada.
- Provenance registrada.
- Deploy permitido apenas com assinatura válida.
- Runtime monitorado.
- Dependências reavaliadas continuamente.

Erros comuns

- Usar latest em imagem.
- Não fixar versão de dependência.
- Baixar artefato sem verificação.
- Não saber quais libs estão em produção.
- Não gerar SBOM.
- Não assinar imagem.
- Pipeline com permissões excessivas.
- Build manual sem rastreabilidade.

Profundidade sênior

O sênior deve proteger o ciclo completo: código → dependência → build → artefato → imagem → registry → deploy → runtime O objetivo é garantir integridade, rastreabilidade, confiança e capacidade de resposta quando uma dependência crítica fica vulnerável.

4. Plano de estudos por módulos

Módulo 1 — Fundamentos de segurança

Objetivo

Construir a base mental para pensar segurança como risco, design e processo.

Conteúdos

- CIA Triad.
- Risco, ameaça, vulnerabilidade e exploit.
- Attack surface.
- Defense in depth.
- Least privilege.
- Secure by design.
- Secure by default.
- Data classification.
- LGPD.
- Privacy by design.

- Threat modeling básico.

Ordem recomendada

- Conceitos fundamentais.
- Classificação de dados.
- Princípios defensivos.
- Threat modeling.
- LGPD e privacidade.

Exercícios técnicos

- Mapear ativos de uma API simples.
- Classificar dados: público, interno, confidencial, sensível.
- Criar um threat model básico para login, cadastro e upload.
- Identificar superfície de ataque de uma aplicação full stack.

Mini projeto

Criar documento de threat modeling da DevCore:

- Ativos.
- Atores.
- Fluxos.
- Riscos.
- Controles.
- Priorização.

Critério de conclusão

Você conclui o módulo quando conseguir explicar os riscos de uma funcionalidade antes de escrever código.

Nível esperado

Básico sólido com transição para intermediário.

Módulo 2 — Aplicação prática em frontend, backend e APIs

Objetivo

Aprender a aplicar segurança diretamente no código.

Conteúdos

- OWASP Top 10.
- OWASP API Security Top 10.
- XSS.
- CSRF.
- CORS.
- SQL Injection.
- NoSQL Injection.
- IDOR.

- BOLA.
- BFLA.
- Mass assignment.
- Rate limiting.
- Error handling seguro.
- Logging seguro.

Ordem recomendada

- OWASP Top 10.
- Segurança frontend.
- Segurança backend.
- Segurança de API.
- Logs e erros.
- Testes de autorização.

Exercícios técnicos

- Corrigir endpoint vulnerável a IDOR.
- Criar DTOs específicos para evitar mass assignment.
- Configurar headers de segurança.
- Implementar validação robusta de entrada.
- Criar testes para acesso negado.
- Adicionar rate limiting em rotas críticas.

Mini projeto

API DevCore segura:

- CRUD de usuários.
- CRUD de projetos.
- Permissão por dono do recurso.
- Logs seguros.
- Tratamento de erro padronizado.
- Rate limit.
- Testes de autorização.

Critério de conclusão

Você conclui o módulo quando conseguir revisar uma API e identificar riscos de autorização, validação, exposição de dados e abuso.

Nível esperado

Intermediário prático.

Módulo 3 — Autenticação, autorização e identidade

Objetivo

Construir um sistema de identidade robusto e seguro.

Conteúdos

- Sessions.
- JWT.
- OAuth2.
- OIDC.
- PKCE.
- Claims.
- Scopes.
- RBAC.
- ABAC.
- MFA.
- Hash de senha.
- Refresh token rotation.
- Token revocation.
- Account logout.
- Rate limiting de login.

Ordem recomendada

- Diferença entre authn e authz.
- Sessions e cookies.
- JWT.
- Refresh token.
- OAuth2/OIDC.
- RBAC.
- ABAC.
- MFA e revogação.

Exercícios técnicos

- Criar login com senha hasheada.
- Implementar access token curto.
- Implementar refresh token rotativo.
- Revogar sessão.
- Proteger cookie com HttpOnly, Secure e SameSite.
- Criar teste para tentativa de reutilização de refresh token.
- Criar roles e permissões.

Mini projeto

Auth seguro na DevCore:

- Login.
- Logout.
- Refresh token rotation.

- Revogação.
- Cookie HttpOnly.
- Proteção CSRF quando aplicável.
- RBAC por endpoint.
- MFA conceitual documentado.

Critério de conclusão

Você conclui o módulo quando conseguir justificar tecnicamente sua estratégia de autenticação e autorização.

Nível esperado

Intermediário forte para avançado.

Módulo 4 — Arquitetura segura e boas práticas

Objetivo

Evoluir de código seguro para arquitetura segura.

Conteúdos

- Secure by design.
- Threat modeling avançado.
- API Gateway.
- Segurança em microsserviços.
- Segurança em monólito modular.
- Zero trust conceitual.
- Segregação de responsabilidades.
- Service-to-service auth.
- Secrets management.
- GraphQL security.
- Observabilidade de segurança.

Ordem recomendada

- Threat modeling por arquitetura.
- Gateway e borda.
- Autorização distribuída.
- Secrets.
- Microsserviços.
- GraphQL.
- Observabilidade.

Exercícios técnicos

- Desenhar arquitetura segura para API + frontend + gateway.
- Definir onde validar autenticação e autorização.
- Criar matriz de permissões.
- Criar política de secrets.

- Criar checklist de revisão arquitetural.

Mini projeto

Desenho arquitetural da DevCore:

- Frontend Next.js.
- Backend Java/Spring.
- PostgreSQL.
- Redis.
- API Gateway.
- Auth service.
- Logs centralizados.
- Pipeline CI/CD.
- Ambiente cloud.

Critério de conclusão

Você conclui o módulo quando conseguir revisar uma arquitetura e defender trade-offs de segurança.

Nível esperado

Avançado.

Módulo 5 — Produção, performance, segurança e escalabilidade

Objetivo

Preparar aplicações para ambientes reais de produção.

Conteúdos

- DevSecOps.
- SAST.
- DAST.
- SCA.
- SBOM.
- Secret scanning.
- Container scanning.
- IaC scanning.
- Cloud IAM.
- KMS.
- WAF.
- Kubernetes RBAC.
- NetworkPolicy.
- Pod Security.
- Audit logs.
- SIEM.

- Detection engineering.
- Incident response.

Ordem recomendada

- Pipeline seguro.
- Scans automatizados.
- Containers.
- Cloud security.
- Kubernetes.
- Logs e detecção.
- Resposta a incidentes.

Exercícios técnicos

- Criar pipeline com SAST e SCA.
- Gerar SBOM.
- Escanear imagem.
- Definir política de bloqueio.
- Criar Dockerfile mais seguro.
- Criar manifesto Kubernetes com security context.
- Criar regra de alerta para login suspeito.
- Criar playbook de incidente.

Mini projeto

DevCore em produção controlada:

- Build automatizado.
- Testes.
- Scans.
- Imagem versionada.
- SBOM.
- Deploy com variáveis seguras.
- Logs estruturados.
- Métricas.
- Alertas.

Critério de conclusão

Você conclui o módulo quando conseguir colocar uma aplicação em produção com controles mínimos de segurança, rastreabilidade e resposta.

Nível esperado

Avançado com base operacional.

Módulo 6 — Domínio sênior e liderança técnica

Objetivo

Desenvolver capacidade de decisão, revisão, mentoria e governança técnica.

Conteúdos

- Security Champion.
- Revisão de arquitetura.
- Risk-based prioritization.
- Definição de políticas.
- SLAs de vulnerabilidade.
- Gestão de exceções.
- Relatórios executivos.
- Métricas de AppSec.
- Supply chain security.
- SLSA.
- Sigstore/Cosign.
- Incident postmortem.
- Mentoria de times.
- Cultura de segurança.

Exercícios técnicos

- Criar política de segurança para PR.
- Criar matriz de risco.
- Definir SLA por severidade.
- Fazer review de arquitetura.
- Criar relatório de vulnerabilidade.
- Criar plano de evolução DevSecOps.
- Conduzir postmortem simulado.

Mini projeto

Programa Security Champion para DevCore:

- Checklist de PR.
- Pipeline seguro.
- Modelo de threat modeling.
- Política de secrets.
- Política de dependências.
- Modelo de relatório.
- Métricas.
- Roadmap de maturidade.

Critério de conclusão

Você conclui o módulo quando conseguir liderar tecnicamente segurança de uma aplicação, orientar outros devs e justificar decisões para engenharia, produto e negócio.

Nível esperado

Sênior inicial em AppSec/Full Stack Security.

5. Critérios de domínio técnico por grande área

Fundamentos e risco

Nível	Critério
Iniciante	Entende CIA, risco, vulnerabilidade e ataque.
Intermediário	Aplica least privilege, defense in depth e classificação de dados.
Avançado	Usa threat modeling para antecipar falhas.
Sênior	Prioriza risco considerando impacto, probabilidade, custo e arquitetura.

OWASP e AppSec

Nível	Critério
Iniciante	Conhece OWASP Top 10.
Intermediário	Corrige falhas comuns no código.
Avançado	Cria testes e controles preventivos.
Sênior	Define padrão de desenvolvimento seguro para o time.

Autenticação e autorização

Nível	Critério
Iniciante	Entende login, sessão, token e permissão.
Intermediário	Implementa JWT, cookie seguro, hashing e RBAC.
Avançado	Implementa refresh rotation, revogação, OAuth2/OIDC e ABAC.
Sênior	Decide arquitetura de identidade baseada em risco e escala.

Frontend security

Nível	Critério
Iniciante	Entende XSS, CSRF e CORS.
Intermediário	Configura headers, cookies e validações.
Avançado	Aplica CSP, reduz risco de dependências e protege uploads.
Sênior	Define modelo seguro de frontend considerando UX, token storage e ameaças reais.

Backend/API security

Nível	Critério
Iniciante	Entende validação, erro seguro e SQL Injection.
Intermediário	Implementa validação, DTOs, rate limit e autorização por recurso.
Avançado	Protege contra BOLA, BFLA, SSRF, mass assignment e abuso.
Sênior	Revisa contratos, domínio, autorização, observabilidade e resiliência.

DevSecOps e supply chain

Nível	Critério
Iniciante	Entende SAST, DAST e SCA.
Intermediário	Configura scans básicos no CI/CD.
Avançado	Usa SBOM, container scanning, secret scanning e policy gates.
Sênior	Protege a cadeia completa com assinatura, provenance, versionamento e governança.

Cloud/Kubernetes

Nível	Critério
Iniciante	Entende IAM, secrets e rede.
Intermediário	Aplica least privilege e segmentação.
Avançado	Configura K8s RBAC, NetworkPolicy e Pod Security.
Sênior	Desenha arquitetura cloud segura, auditável, escalável e economicamente viável.

Detecção e resposta

Nível	Critério
Iniciante	Entende logs e alertas.
Intermediário	Cria logs úteis e correlação simples.
Avançado	Usa MITRE ATT&CK, tuning e playbooks.
Sênior	Conecta detecção, resposta, negócio, postmortem e melhoria contínua.

6. Aplicação prática em projetos reais

APIs

Aplicar:

- Autorização por recurso.
- Rate limiting.

- DTOs específicos.
- Validação de entrada.
- Erros padronizados.
- Logs sem dados sensíveis.
- Testes de autorização.

Banco de dados

Aplicar:

- Queries parametrizadas.
- Controle de permissões.
- Criptografia quando necessário.
- Mascaramento de dados.
- Backup seguro.
- Migrações revisadas.
- Auditoria de alterações sensíveis.

Microserviços

Aplicar:

- Autenticação service-to-service.
- Escopos por serviço.
- Gateway.
- Segregação de responsabilidades.
- Secrets por serviço.
- Observabilidade distribuída.
- Autorização consistente.

Mensageria

Aplicar:

- Controle de acesso a tópicos.
- Criptografia em trânsito.
- Não enviar dados sensíveis desnecessários.
- Validação de schema.
- DLQ segura.
- Idempotência.
- Auditoria de eventos críticos.

Segurança

Aplicar:

- Threat modeling.
- OWASP ASVS.
- Revisão de PR.
- Pentest controlado.
- Hardening.

- Políticas de secrets.
- Gestão de vulnerabilidades.

Observabilidade

Aplicar:

- Logs estruturados.
- Correlation ID.
- Métricas de autenticação.
- Alertas de abuso.
- Dashboards de segurança.
- Playbooks.

Testes

Aplicar:

- Testes unitários de autorização.
- Testes de validação.
- Testes de erro.
- Testes de rate limit.
- Testes de integração com roles.
- DAST em ambiente controlado.

Deploy

Aplicar:

- Pipeline com gates.
- SBOM.
- Imagem assinada.
- Secrets externos.
- Deploy rastreável.
- Rollback seguro.
- Aprovação para mudanças críticas.

Arquitetura

Aplicar:

- Defense in depth.
- Least privilege.
- Segurança por domínio.
- Separação de camadas.
- Gateway.
- Políticas de acesso.
- Revisão de riscos.

Performance

Aplicar:

- Rate limit sem bloquear usuários legítimos.

- Hash de senha com custo adequado.
- Logs úteis sem excesso.
- CSP sem quebrar carregamento.
- WAF com regras calibradas.
- Query complexity em GraphQL.

Manutenção evolutiva

Aplicar:

- Atualização de dependências.
- Refatoração segura.
- Documentação de exceções.
- Roadmap de débito técnico de segurança.
- Revisão contínua de permissões.

7. Cronograma sugerido — 20 semanas

Semanas 1-2: Fundamentos

Prioridade alta.

Estudar:

- CIA Triad.
- Risco.
- Vulnerabilidade.
- Threat modeling.
- Attack surface.
- LGPD.
- Privacy by design.

Entrega:

Threat model inicial da DevCore.

Checkpoint:

Explicar riscos de login, upload e consulta de dados.

Semanas 3-5: OWASP, frontend e backend security

Prioridade alta.

Estudar:

- OWASP Top 10.
- OWASP API Top 10.
- XSS.
- CSRF.
- CORS.
- SQL/NoSQL Injection.
- IDOR/BOLA.

- Mass assignment.
- Logging seguro.

Entrega:

API com validação, autorização por recurso e erros seguros.

Checkpoint:

Revisar 5 endpoints e apontar riscos.

Semanas 6-8: Autenticação e autorização

Prioridade crítica.

Estudar:

- Sessions.
- JWT.
- Refresh token rotation.
- OAuth2.
- OIDC.
- PKCE.
- RBAC.
- ABAC.
- MFA.
- Hashing.

Entrega:

Auth seguro na DevCore.

Checkpoint:

Demonstrar fluxo de login, refresh, revogação e permissão.

Semanas 9-11: Arquitetura segura

Prioridade alta.

Estudar:

- Secure by design.
- Gateway.
- Microserviços.
- Secrets.
- Service-to-service auth.
- GraphQL security.
- Observabilidade de segurança.

Entrega:

Diagrama arquitetural seguro da DevCore.

Checkpoint:

Justificar onde cada controle de segurança fica.

Semanas 12-14: DevSecOps

Prioridade alta.

Estudar:

- SAST.
- DAST.
- SCA.
- SBOM.
- Secret scanning.
- Container scanning.
- IaC scanning.
- Policy gates.

Entrega:

Pipeline seguro com checks automatizados.

Checkpoint:

Definir o que bloqueia build e o que vira backlog.

Semanas 15-17: Cloud, containers e Kubernetes

Prioridade média/alta.

Estudar:

- IAM.
- KMS.
- Secrets Manager.
- Security groups.
- WAF.
- Kubernetes RBAC.
- Pod Security.
- NetworkPolicy.
- Audit logs.
- Image signing.

Entrega:

Deploy seguro em ambiente controlado.

Checkpoint:

Explicar permissões, rede, secrets e runtime security.

Semana 18: Pentest controlado

Prioridade média.

Estudar:

- OWASP ZAP.
- Burp Suite CE conceitual.
- Teste de autenticação.

- Teste de autorização.
- Misconfiguration.
- Relatório de vulnerabilidade.

Entrega:

Relatório de pentest da própria DevCore.

Checkpoint:

Evidência, impacto, severidade, correção e reteste.

Semana 19: SIEM, detecção e resposta

Prioridade média/alta.

Estudar:

- Logs de segurança.
- Correlation rules.
- MITRE ATT&CK.
- Alert tuning.
- Incident response.
- Playbooks.

Entrega:

Dashboard e playbook de incidente.

Checkpoint:

Simular evento suspeito e explicar resposta.

Semana 20: Supply chain e avaliação sênior

Prioridade alta.

Estudar:

- SBOM.
- SLSA.
- Sigstore.
- Cosign.
- Provenance.
- Dependency pinning.
- Artifact registry.
- Policy gates.

Entrega:

Roadmap de maturidade Security Champion.

Checkpoint:

Apresentar defesa técnica da arquitetura completa.

8. Projetos recomendados para portfólio

Projeto 1 — DevCore Auth Seguro

Inclui:

- Login.
- Senha com Argon2/bcrypt.
- JWT curto.
- Refresh token rotativo.
- Cookie HttpOnly.
- CSRF.
- RBAC.
- Revogação.
- Logs de segurança.

Projeto 2 — API Segura com OWASP

Inclui:

- CRUD protegido.
- Validação.
- DTOs.
- Autorização por recurso.
- Rate limit.
- Tratamento de erro.
- Testes de segurança.

Projeto 3 — Pipeline DevSecOps

Inclui:

- SAST.
- SCA.
- Secret scanning.
- Container scanning.
- SBOM.
- Policy gate.
- Relatório de vulnerabilidade.

Projeto 4 — Deploy Cloud/Kubernetes Seguro

Inclui:

- IAM mínimo.
- Secrets externos.
- NetworkPolicy.
- Pod Security.
- Logs.

- Imagem assinada.
- Auditoria.

Projeto 5 — Relatório de Pentest Controlado

Inclui:

- Escopo.
- Metodologia.
- Vulnerabilidades.
- Evidências.
- Severidade.
- Correção.
- Reteste.

9. Checklist de domínio técnico

Fundamentos

- ☐ Explica CIA Triad.
- ☐ Diferencia risco, vulnerabilidade, ameaça e exploit.
- ☐ Faz threat modeling.
- ☐ Classifica dados.
- ☐ Aplica least privilege.
- ☐ Projeta secure by default.

OWASP

- ☐ Conhece OWASP Top 10.
- ☐ Conhece OWASP API Top 10.
- ☐ Corrige Broken Access Control.
- ☐ Corrige Injection.
- ☐ Corrige Authentication Failures.
- ☐ Corrige Security Misconfiguration.

Auth

- ☐ Implementa login seguro.
- ☐ Usa hash de senha correto.
- ☐ Implementa refresh token rotation.
- ☐ Implementa revogação.
- ☐ Usa cookie seguro.
- ☐ Aplica RBAC.
- ☐ Entende ABAC.
- ☐ Entende OAuth2/OIDC/PKCE.

Frontend

- ☐ Reduz risco de XSS.
- ☐ Configura CSP.
- ☐ Configura CORS corretamente.
- ☐ Protege contra CSRF.
- ☐ Evita secrets no bundle.
- ☐ Avalia risco de LocalStorage.

Backend/API

- ☐ Valida entrada.
- ☐ Evita SQL/NoSQL Injection.
- ☐ Evita IDOR/BOLA.
- ☐ Evita mass assignment.
- ☐ Implementa rate limiting.
- ☐ Protege logs.
- ☐ Trata erros com segurança.

DevSecOps

- ☐ Usa SAST.
- ☐ Usa DAST.
- ☐ Usa SCA.
- ☐ Gera SBOM.
- ☐ Escaneia secrets.
- ☐ Escaneia containers.
- ☐ Define policy gates.

Cloud/Kubernetes

- ☐ Aplica IAM mínimo.
- ☐ Usa KMS/Secrets Manager.
- ☐ Segmenta rede.
- ☐ Configura WAF.
- ☐ Usa Kubernetes RBAC.
- ☐ Usa Pod Security.
- ☐ Usa NetworkPolicy.
- ☐ Habilita audit logs.

SIEM e resposta

- ☐ Cria logs úteis.
- ☐ Evita dados sensíveis nos logs.
- ☐ Cria alertas.
- ☐ Usa MITRE ATT&CK como referência.
- ☐ Cria playbooks.
- ☐ Faz postmortem.

Supply chain

- ☐ Fixa dependências.
- ☐ Gera SBOM.
- ☐ Assina imagens.
- ☐ Usa registry confiável.
- ☐ Valida provenance.
- ☐ Bloqueia deploy inseguro.

10. Indicadores de prontidão para nível avançado/sênior

Você está pronto para atuar em nível avançado/sênior nessa trilha quando conseguir:

- Projetar autenticação e autorização sem depender apenas de tutorial.
- Identificar falhas de autorização em APIs reais.
- Revisar PRs apontando riscos concretos.
- Explicar trade-offs entre JWT, session e opaque token.
- Criar threat model de uma funcionalidade crítica.
- Implementar pipeline com gates de segurança.
- Priorizar vulnerabilidades por risco, não apenas por score.
- Proteger secrets, dependências, imagens e deploy.
- Criar relatório de vulnerabilidade claro e acionável.
- Configurar logs úteis para investigação.
- Defender decisões técnicas para devs, arquitetura, produto e negócio.
- Liderar práticas de segurança dentro de um time de desenvolvimento.

Resumo final da trilha

A Trilha Segurança Full Stack forma um desenvolvedor capaz de atuar como ponte entre desenvolvimento, arquitetura, infraestrutura e segurança. O objetivo final é sair do nível de “conheço vulnerabilidades” para “projeto, implemento, reviso, automatizo, monitoro e evoluo aplicações seguras em produção”. A evolução ideal segue esta ordem:

- Fundamentos e risco.
- OWASP e segurança aplicada.
- Autenticação e autorização.
- Segurança frontend/backend/API.

- Arquitetura segura.
- DevSecOps.
- Cloud e Kubernetes.
- Pentest controlado.
- Detecção e resposta.
- Supply chain security.
- Liderança técnica como Security Champion.